

Microservices Distributed Caching

Introduction: What is Caching, Types and Distributed Caching

Il problema ereditato

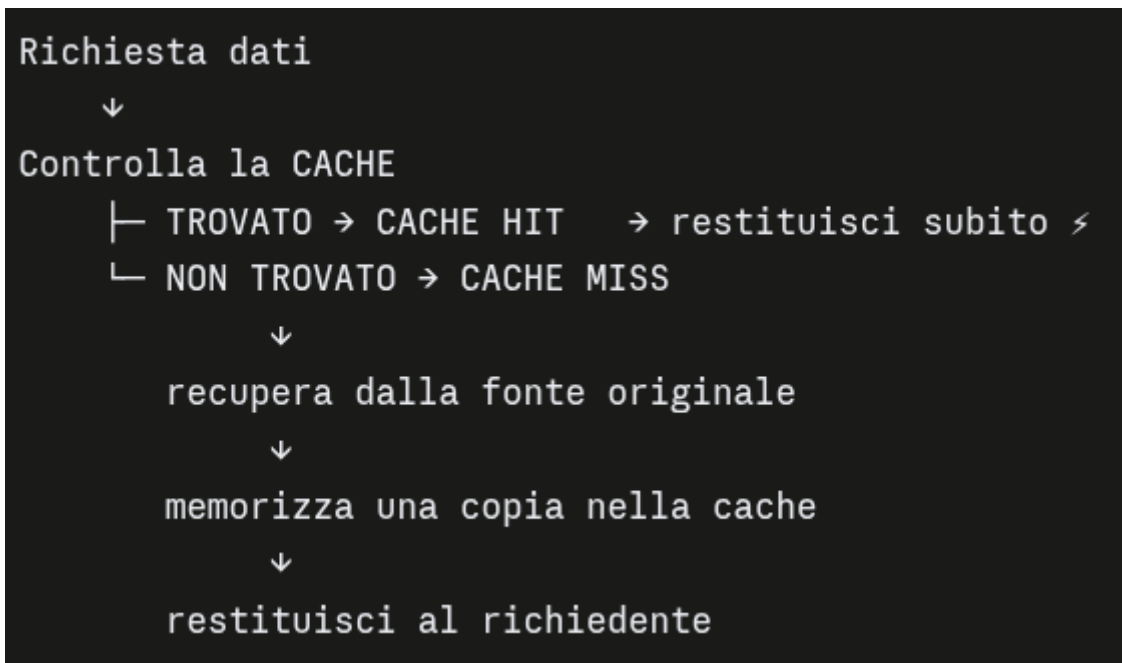
"Il costo dell'accesso frequente al data store. Anche con eventing e transazioni ottimizzate, le operazioni di database sono spesso costose e lente. Il nostro obiettivo è rendere i microservizi più veloci e ridurre il carico sui data store primari."

Definizione di caching

Una tecnica per migliorare le prestazioni del sistema memorizzando copie dei dati ad accesso frequente in una posizione di archiviazione temporanea ad alta velocità: la cache.

"L'obiettivo principale è servire le richieste successive per gli stessi dati direttamente dalla cache, piuttosto che eseguire ripetutamente l'operazione originale più costosa — interrogare un database, chiamare un'API esterna, o eseguire un calcolo complesso."

Il meccanismo:



I 4 benefici:

"Aumenta significativamente le prestazioni, riducendo la latenza, migliorando la scalabilità e riducendo il carico sui sistemi di backend. E può persino migliorare la disponibilità, consentendo all'applicazione di servire i dati dalla cache se la fonte originale non è temporaneamente disponibile."

"Questo è particolarmente vero per i dati del catalogo prodotti, che non cambiano molto frequentemente."

★ I 3 tipi di architettura di caching

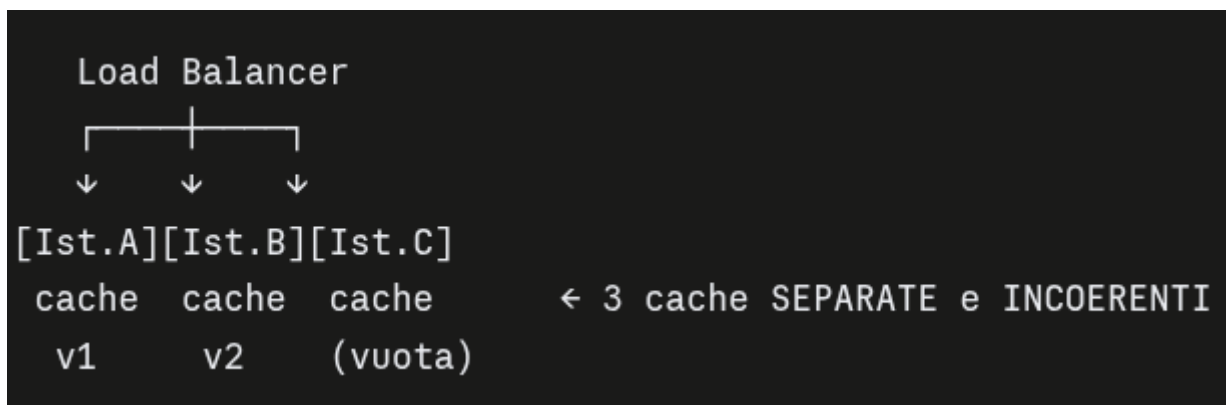
Tipo	Dove vive	Velocità	Problemi
In-memory cache	"Direttamente nella RAM del server applicativo"	Massima	⚠ "I dati sono volatili : si perdono al riavvio del processo" + ⚠ "Ogni istanza avrà la sua cache separata, il che non è ideale con più istanze"
Disk cache	"Su supporto persistente — hard disk o SSD"	Media	"Più lente della memoria, ma offrono persistenza tra i riavvii . I browser la usano per le risorse web"
Distributed cache ★	"Un servizio o cluster di caching condiviso ESTERNO , accessibile da più server applicativi o istanze"	Alta	Richiede un hop di rete

★ Perché la cache distribuita è essenziale nei microservizi

"I nostri microservizi sono progettati per essere **stateless e scalati orizzontalmente**: eseguiamo **più istanze di ogni servizio dietro un load balancer**."

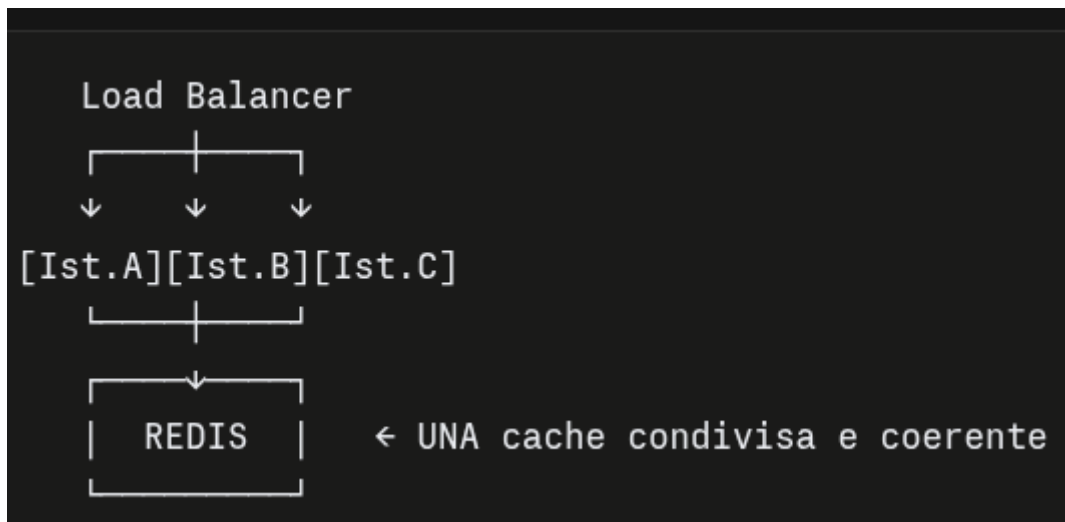
Il problema con la cache in-memory in questo scenario:

"Se ogni istanza usa la propria cache in-memory, **ogni istanza avrebbe una versione diversa e potenzialmente incoerente dei dati**."



"Una richiesta gestita **dall'istanza A** potrebbe ottenere una versione in cache, mentre una successiva richiesta degli stessi dati gestita **dall'istanza B**, che non ha quella copia locale, dovrebbe rivolgersi al database."

La soluzione:



"Una cache distribuita fornisce un livello di caching centralizzato e condiviso che TUTTE le istanze possono utilizzare. Inoltre, la cache stessa può essere scalata indipendentemente dai microservizi."

Nota il richiamo esplicito alla Lezione 4: il requisito "stateless" dello scaling orizzontale è esattamente ciò che rende necessaria la cache distribuita.

Caching Strategies in Distributed Caching

*"Quando integriamo una cache distribuita come Redis, dobbiamo decidere **come il nostro codice applicativo la userà**. Questo modo è definito dalla **strategia di caching**."*

*"La strategia migliore dipende dalle **caratteristiche dei dati**: frequenza di accesso, frequenza di modifica, e le esigenze specifiche dell'applicazione."*

★ Le 4 strategie

1. Cache-Aside (Lazy Loading) — la più comune

Il codice dell'applicazione è responsabile di: controllare la cache, recuperare dal database primario in caso di miss, e aggiungere esplicitamente i dati alla cache.

"I dati vengono caricati pigramente (lazily) nella cache solo quando vengono richiesti la prima volta e non vengono trovati."

L'applicazione ha il controllo esplicito.

```
App —1. controlla→ [Cache]
App —2. se miss→ [Database]
App —3. scrive→ [Cache]
```

2. Read-Through

La cache si colloca **TRA** l'applicazione e il database per le letture.

*"L'applicazione si rivolge **SEMPRE** alla cache. In caso di cache miss, è la **CACHE STESSA** responsabile di caricare i dati mancanti dal database, popolarli, e restituirli all'applicazione."*

App → [Cache] —(se miss, autonomamente)—→ [Database]

"La logica di caricamento dei dati è astratta dall'applicazione."

Differenza chiave con cache-aside: chi gestisce il miss. Nel cache-aside è l'app; nel read-through è la cache.

3. Write-Through

I dati vengono scritti nella cache e poi inviati **SINCRONAMENTE** al database primario.

*"Scrivendo i dati solo nella cache, il sistema di cache è responsabile di scrivere immediatamente i dati nel database sottostante, e garantisce che cache e database siano **SEMPRE** coerenti dopo una scrittura."*

Vantaggio

Svantaggio

*"I dati nella cache sono **sempre** aggiornati" "Aggiunge latenza alle operazioni di scrittura"*

4. Write-Behind (Write-Back)

I dati vengono scritti nella cache, che li riconosce **IMMEDIATAMENTE**, e poi aggiorna il database primario in modo **ASINCRONO** dopo un ritardo o in batch.

*"La cache **accoda** le modifiche e le scrive nel database in background."*

Vantaggio

Svantaggio

*"Le operazioni di scrittura sono molto veloci per l'applicazione"  **"Rischio di PERDITA DI DATI se la cache si rompe prima che i dati vengano scritti sul database. I dati del database sono eventually consistent con la cache"***

Riepilogo

Strategia	Chi gestisce il miss/la scrittura	Coerenza	Velocità scrittura
Cache-Aside	L'applicazione	Eventual	—
Read-Through	La cache	Eventual	—
Write-Through	La cache (sincrono)	 Forte	 Lenta
Write-Behind	La cache (asincrono)	 Eventual	 Velocissima (ma rischio perdita dati)

Cache-Aside Pattern for Microservices

Definizione

Chiamato anche lazy loading, perché i dati vengono caricati nella cache solo quando sono effettivamente necessari e non vengono trovati.

"La logica dell'applicazione dentro il microservizio è responsabile di controllare la cache, recuperare dal database, e poi popolare la cache. La cache si trova A LATO (aside) del data store primario."

Il flusso completo

```
1. [Catalog Service] chiede a Redis: "hai il prodotto 123?"
   |
   |— SÌ (CACHE HIT) —→ recupera da Redis → restituisce ✨ velocissimo
   |
   |— NO (CACHE MISS)
       ↓
       2. interroga PostgreSQL
         ↓
         3. MEMORIZZA una copia in Redis
           ↓
           4. restituisce i dati al chiamante
```

*"Alcuni sistemi di cache offrono capacità read-through integrate, ma con il cache-aside il codice dell'applicazione gestisce **ESPLICITAMENTE** queste interazioni."*

I 4 vantaggi

Vantaggio	Dettaglio
Riduce il carico del database ★	<i>"Riduciamo drasticamente il numero di query di lettura che colpiscono il database primario. Si evita che il database diventi un collo di bottiglia."</i>
Tempi di risposta e latenza	<i>"Recuperare da una cache distribuita in-memory è di ordini di grandezza più veloce rispetto a un database su disco"</i>
Scalabilità	<i>"Poiché il carico è ridotto, il microservizio e il suo database gestiscono un volume maggiore. Anche la cache stessa può essere scalata indipendentemente"</i>
Controllo dell'applicazione ★	<i>"Il codice ha controllo esplicito su QUALI dati vengono memorizzati, COME vengono generate le chiavi, e la DURATA della cache"</i>

★ Le 5 sfide

1. Invalidazione della cache — "la sfida più importante"

"Se i dati nel database primario vengono aggiornati, la copia in cache diventa STALE (obsoleta). Se non abbiamo un meccanismo per aggiornare o rimuovere dalla cache i dati non aggiornati, l'applicazione potrebbe fornire informazioni errate."

2. Cache miss penalty ★

"La PRIMA richiesta di un dato non ancora in cache sarà in realtà LEGGERMENTE PIÙ LENTA rispetto all'assenza di cache, perché l'applicazione deve prima controllare la cache, poi andare al database, e quindi scrivere nella cache."

Un punto onesto e spesso trascurato: **il caching non è gratis nemmeno in lettura**. Paghi un piccolo sovrapprezzo sul primo accesso.

3. Thundering Herd / Cache Stampede ★

"Se un elemento della cache molto popolare SCADE, molte richieste simultanee per quell'elemento possono mancare la cache nello stesso momento e cercare di raggiungere il database CONTEMPORANEAMENTE per recuperare e ripopolare. Questo può sovraccaricare il database."

Le mitigazioni citate: *"scadenze scaglionate (staggered expiration)"* o *"popolamento della cache basato sui log"*.

Come funziona la mitigazione più comune: invece di dare a 10.000 chiavi lo stesso TTL di 5 minuti (scadono tutte insieme), si aggiunge un **jitter** casuale — TTL tra 4:30 e 5:30 — così le scadenze si distribuiscono nel tempo. Altra tecnica: un **lock** che permette a **una sola** richiesta di ricaricare dal database mentre le altre attendono o servono il valore vecchio.

4. Complessità del codice applicativo

*"La logica per controllare, recuperare e scrivere **risiede nel codice dell'applicazione**. Aggiunge complessità rispetto, ad esempio, a una read-through cache dove **il sistema di cache gestisce automaticamente i miss**."*

5. TTL e politiche di eviction

"Per quanto tempo i dati devono rimanere nella cache prima di scadere? Cosa succede quando la cache è piena? Quali oggetti vengono rimossi (evicted)? Sono decisioni cruciali di configurazione."

Design: Distributed Caching with Cache-Aside Pattern

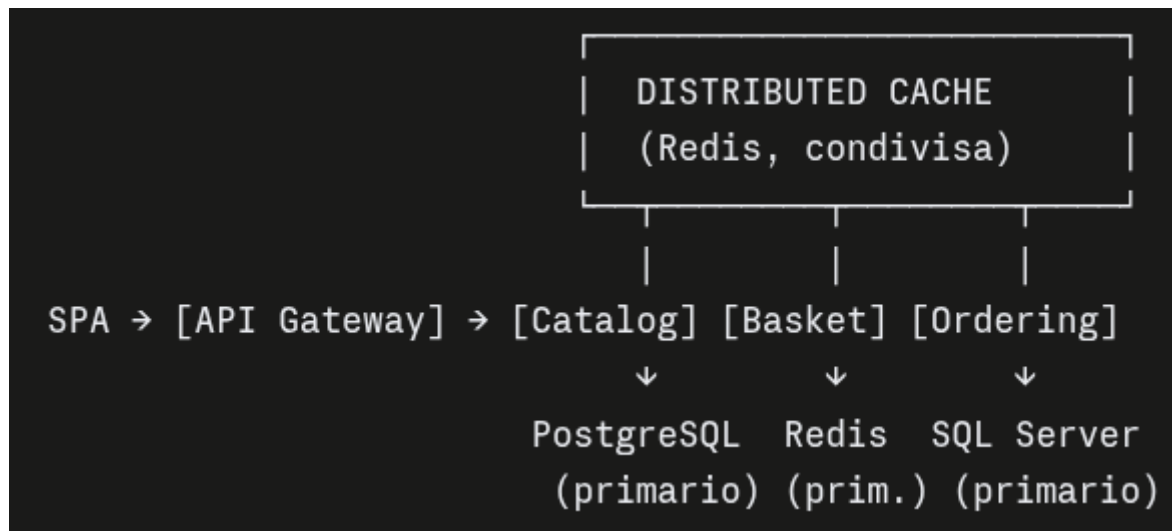
Toolkit aggiornato

Nuovi requisiti non funzionali:

1. **Ridurre la latenza** per le operazioni di lettura frequenti tra i vari servizi
2. **Diminuire il carico sui database primari**, servendo le richieste comuni da un livello più veloce
3. **Migliorare reattività e throughput** complessivi

Nuovi pattern: Cache-Aside con tecnologia di cache distribuita + la consapevolezza che servono strategie per **invalidazione, coerenza e politiche di eviction**.

Il design — livello 1: cache nei microservizi



"Introduciamo **un'istanza di cache distribuita condivisa e centralizzata** nella nostra infrastruttura di backend, **accessibile da tutti i microservizi**."

"Quando Catalog riceve una richiesta, **prima controlla la cache**. Se è un **hit**, recupera i dati direttamente e li restituisce — **il database primario NON viene toccato**. Se è un **miss**, interroga il proprio database, **scrive una copia nella cache con un tempo di scadenza appropriato**, e restituisce i dati."

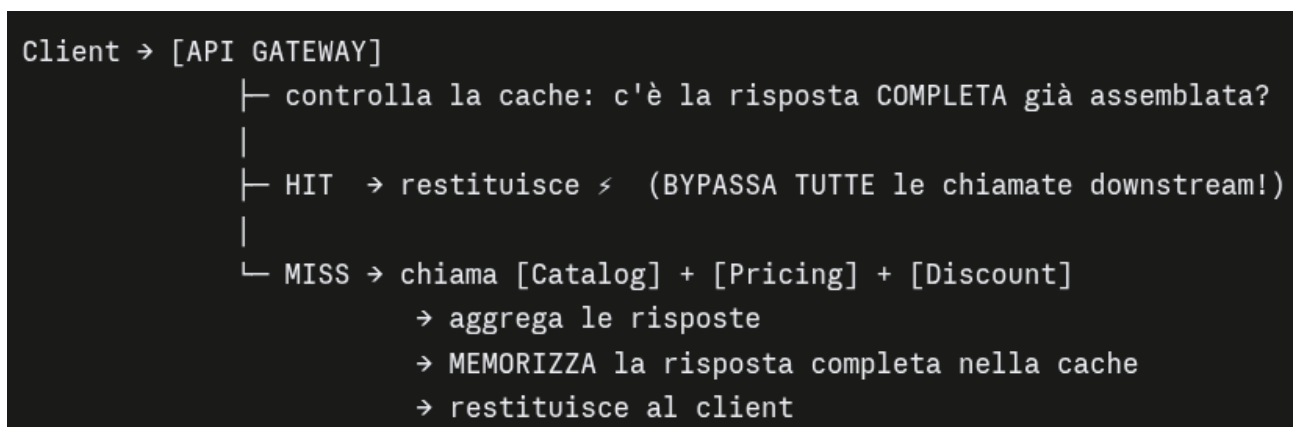
"Questo pattern può essere applicato a **tutti i microservizi che gestiscono dati letti di frequente, relativamente costosi da recuperare o calcolare**."

Il design — livello 2: cache nell'API Gateway

Questa è la parte più interessante della sezione.

"Vale la pena notare che il **cache-aside non è solo per i singoli microservizi**. Può anche essere implementato **efficacemente DENTRO L'API GATEWAY stesso**, soprattutto se il gateway esegue l'**aggregazione**."

Ricorda il **Gateway Aggregation** della Lezione 8.



★ "Se è un **cache hit**, il gateway **recupera la risposta completa direttamente da Redis e la restituisce al client, BYPASSANDO TUTTE le chiamate ai microservizi a valle**."

"La **cache key** potrebbe basarsi sull'**URL o sui parametri della richiesta**."

*"Questo approccio **riduce ulteriormente il carico sui microservizi di backend.**"*

Perché è potente: cachare a livello di gateway salva **N chiamate di rete + N query** in un colpo solo, invece di una sola query. È la forma di caching col rapporto beneficio/costo più alto — ma anche quella con il rischio di staleness più diffuso, perché la risposta aggregata diventa stale se **anche uno solo** dei servizi sottostanti cambia.

Scelte tecnologiche

Opzione	Note
Redis ★	<i>"Archivio dati in-memory open source rinomato per velocità e flessibilità. Supporta diverse strutture dati oltre alle semplici coppie chiave-valore. Ampiamente usato nelle architetture a microservizi"</i>
Memcached	<i>"Altro popolare key-value store in-memory open source. Tipicamente più semplice di Redis, ma molto efficace e veloce per la memorizzazione di oggetti"</i>
Cloud gestiti	<i>AWS ElastiCache (supporta sia Redis sia Memcached), Azure Cache for Redis, Google Cloud Memorystore — "gestiscono patching, scaling e backup, consentendo di concentrarsi sull'uso della cache"</i>

I 5 svantaggi

1. Complessità dell'invalidazione — *"la sfida più grande"*

"Quando i dati originali cambiano, come si garantisce che i dati corrispondenti nella cache vengano rimossi o aggiornati? Se non gestita correttamente, l'applicazione serve informazioni non aggiornate."

2. Hop di rete aggiuntivo

*"L'accesso a una cache distribuita **comporta comunque una chiamata di rete.** Pur essendo molto più veloce di una chiamata al database, **non è veloce quanto un accesso alla memoria in-process.**"*

3. Costo e gestione del cluster

"È un altro pezzo di infrastruttura da fornire, pagare, monitorare e scalare."

4. Coerenza tra cache e database

*"I dati nella cache **sono una copia.** I client potrebbero **temporaneamente vedere dati obsoleti.** Dobbiamo abbracciare l'**eventual consistency.**"*

5. Overhead di serializzazione

*"I dati devono essere **serializzati prima di essere memorizzati e deserializzati quando recuperati.** Aggiunge un po' di overhead di CPU."*

*"Nonostante queste considerazioni, per molti carichi di lavoro read-heavy nei microservizi, i guadagni di prestazioni di una cache distribuita ben implementata sono **INDISPENSABILI.**"*

Code: Distributed Caching with Cache-Aside Pattern

Cartella 10-eshop-microservices-hybrid-cache. Il pattern è implementato nel microservizio **Basket**.

★ **Il cambiamento strutturale: Basket ora ha un database persistente**

```
// BACKING SERVICES

var basketDb = postgres.AddDatabase("basketdb"); // ← NUOVO: DB primario
var cache = builder.AddRedis("cache").WithRedisInsight();

// MICROSERVIZI

builder.AddProject<Basket>("basket")
    .WithReference(basketDb) // fonte di verità
    .WithReference(cache) // cache
    .WithReplicas(3); // ← 3 istanze!
```

Perché il cambio:

"In precedenza usavamo SOLO la cache Redis per i dati del microservizio Basket. Ma questa volta abbiamo aggiunto il database persistente. Perché?"

★ *"Se usi Redis SOLO per i dati del carrello, la cache Redis può essere eliminata o cancellata — e tutti i dati del carrello andrebbero persi. Ora Redis sarà la nostra cache, ma il basket DB sarà la fonte PERSISTENTE DI VERITÀ."*

Questo è un cambiamento architetturale importante: nelle lezioni precedenti Redis era il *database* di Basket (scelta polyglot). Ora Redis torna al suo ruolo naturale: **cache**, con un database relazionale sotto.

Le 3 repliche:

"Ho anche aggiunto `WithReplicas(3)` per il microservizio Basket, per dimostrare come funziona la cache distribuita con istanze multiple."

★ **La configurazione: Hybrid Cache**

In `Basket/Program.cs`:

```
// database primario

builder.AddNpgsqlDbContext<BasketDbContext>("basketdb");

// cache distribuita Redis

builder.AddRedisDistributedCache("cache");
```

// HYBRID CACHE — la parte chiave

```
builder.Services.AddHybridCache(options =>
{
    options.DefaultEntryOptions = new HybridCacheEntryOptions
    {
        Expiration    = TimeSpan.FromMinutes(5), // TTL su Redis (L2)
        LocalCacheExpiration = TimeSpan.FromMinutes(1), // TTL in-memory (L1)
        Flags = HybridCacheEntryFlags.DisableLocalCache // ← per la demo
    };
});
```

Definizione: HybridCache — *"la HybridCache in .NET può gestire SIA una cache locale in-memory di PRIMO LIVELLO (L1) SIA una cache distribuita di SECONDO LIVELLO (L2) come Redis."*

Richiesta

↓

[L1: in-memory] ← velocissima, per-istanza

↓ miss

[L2: Redis] ← veloce, condivisa

↓ miss

[PostgreSQL] ← fonte di verità

Il flag della demo:

*"Per questa dimostrazione, per essere molto chiari quando stiamo colpendo Redis, ho impostato il flag **DisableLocalCache**. Questo forza la hybrid cache ad andare **SEMPRE** su Redis."*

Un dettaglio didattico onesto: la L1 renderebbe la demo illeggibile, perché i cache hit non arriverebbero mai fino a Redis.

Il cache-aside in una riga: GetOrCreateAsync

In Basket/Services/BasketService.cs:

```
public async Task<ShoppingCart> GetBasket(string userName)
{
    return await cache.GetOrCreateAsync(
        userName, // ← la CACHE KEY
        async token => // ← il FACTORY DELEGATE
```

```

{
    // eseguito SOLO in caso di CACHE MISS
    return await dbContext.ShoppingCarts
        .Include(c => c.Items)
        .FirstOrDefaultAsync(c => c.UserName == userName, token);
});
}

```

"Questo metodo implementa lo schema cache-aside, perché esegue:"

1. **"Controlla la cache distribuita per trovare i dati del carrello con lo username come chiave"**
2. **"Se trovato, restituisce i dati dalla cache Redis"**
3. **"Se non trovato — cache miss — viene eseguito il FACTORY DELEGATE, che recupera i dati dal database primario PostgreSQL"**
4. **"Il risultato del factory viene automaticamente memorizzato in Redis dalla hybrid cache prima di essere restituito"**

Nota quanto è elegante: GetOrCreateAsync incapsula tutti e 4 i passi del cache-aside in una sola chiamata, eliminando la sfida n. 4 ("complessità del codice applicativo"). L'applicazione mantiene il controllo su chiave e TTL, ma non scrive il boilerplate.

Il lato scrittura: UpdateBasket (write-through)

```

public async Task<ShoppingCart> UpdateBasket(ShoppingCart basket)
{
    // 1. SEMPRE scrivi prima nel DATA STORE PRIMARIO
    var existing = await dbContext.ShoppingCarts...;
    if (existing != null) { /* aggiorna */ }
    else { dbContext.ShoppingCarts.Add(basket); }
    await dbContext.SaveChangesAsync();

    // 2. POI aggiorna esplicitamente la cache
    await cache.SetAsync(basket.UserName, basket); // ← write-through

    return basket;
}

```

*"Nel metodo `UpdateBasket` scriviamo **SEMPRE** le modifiche nel nostro data store primario. Dopo aver salvato con successo il database, usiamo esplicitamente l'oggetto cache per aggiornarla con `SetAsync`, passando la chiave (username) e l'oggetto `JSON ShoppingCart`."*

★ *"In questo modo ci assicuriamo che la cache contenga la versione più recente dopo un aggiornamento. Questo è talvolta chiamato **write-through** ed è parte della strategia completa del pattern **cache-aside**."*

Nota la combinazione: letture in **cache-aside** (lazy), scritture in **write-through** (eager). È la combinazione più comune in produzione, e risolve in gran parte il problema dell'invalidazione: la cache non diventa mai stale *per effetto delle tue scritture* — solo per effetto di quelle altrui.

★ La demo end-to-end

Stato iniziale: Redis Insight → **nessuna chiave**. pgAdmin → tabella `shopping_carts` **vuota**.

Fase 1 — Popolamento (scritture)

1. Si aggiungono al carrello **bastoncini da trekking** → viene creato un nuovo carrello → **aggiornato nella cache**
2. Si aggiunge un secondo articolo → aggiorna il carrello esistente → **aggiorna Redis**
3. **Verifica pgAdmin:** il carrello è nel database, con **2 righe** in `cart_items` ✓
4. **Verifica Redis Insight:** una **chiave** con il carrello completo in **formato JSON** ✓

Fase 2 — Simulazione del CACHE MISS ★

*"Per simulare il **cache-aside**, **cancellerò questa chiave dalla cache distribuita**. Così possiamo vedere come sia possibile **ripopolarla recuperando dal database**."*

5. Si **cancella la chiave** da Redis Insight
6. Si apre la pagina del carrello → chiama `GetBasket` → **CACHE MISS**

La traccia Aspire mostra:

```
[Basket Service]
1. controlla la cache → NESSUN elemento ← CACHE MISS
2. SELECT su PostgreSQL (carrello + articoli)
3. aggiorna le informazioni nella cache Redis
```

Fase 3 — Il CACHE HIT ★

7. Si **ricarica la pagina** — entro i 5 minuti di TTL

La traccia mostra:

```
WebApp → [API Gateway] → [Basket Service, istanza N]
                                   └─ recupera da REDIS
```

✗ NESSUNA operazione sul database PostgreSQL

★ *"Questa volta GetBasket trova i dati in Redis e li restituisce molto più velocemente, SENZA colpire il database. Questa è la nostra operazione di cache hit, senza visitare le chiamate al database."*

8. Verifica finale: **i record sono ancora nel database** (fonte di verità intatta) **e la chiave è tornata in Redis** ✓

"Abbiamo implementato con successo il pattern cache-aside nel microservizio Basket, usando la HybridCache supportata dalla cache distribuita Redis."

Problem: Deploy Microservices Anytime with Zero-downtime and Flexible Scale

Dove siamo arrivati

"La nostra applicazione non è più una singola applicazione. È una FLOTTA di microservizi specializzati: Catalog, Basket, Ordering, Payment, Inventory, Notification, il gateway API, e potenzialmente più BFF."

"Ogni servizio è di proprietà di un team dedicato, che itera e sviluppa funzionalità in modo indipendente."

La pressione del business

"L'azienda si aspetta che questi team implementino immediatamente le nuove funzionalità per competere sul mercato, senza aspettare date di deployment coordinate."

"Inoltre, l'eShop deve gestire milioni di richieste, soprattutto durante i picchi come il Black Friday, e i diversi servizi avranno esigenze di scalabilità diverse in momenti diversi."

★ La domanda

"Questo è fantastico dal punto di vista dell'agilità e dell'architettura, ma dal punto di vista OPERATIVO, come lo gestiamo?"

"Se distribuire ogni aggiornamento a ogni microservizio, o scalare ogni servizio, è un processo MANUALE complesso, perderemo rapidamente tutti i vantaggi di agilità che abbiamo progettato."

Questa è l'osservazione chiave: **l'agilità architetturale è inutile senza agilità operativa.**

Le 4 considerazioni

Considerazione	Dettaglio
Deployment senza downtime	"L'eShop deve essere disponibile 24/7 . Non possiamo permetterci di mettere offline i servizi per distribuire nuove versioni. Significherebbe perdite di fatturato e utenti frustrati "
Scalabilità agile e flessibile ★	"Durante il Black Friday, Ordering o Discount potrebbero aver bisogno di scalare fino a 10 VOLTE la capacità normale quasi istantaneamente, per poi ridimensionarsi quando il picco passa per risparmiare sui costi. Deve essere dinamico e reattivo, idealmente con operazioni di scaling AUTOMATIZZATE "
Complessità di molte parti mobili	"Tenere traccia di quale versione è in esecuzione su quale server, gestire le configurazioni, assicurarsi che si trovino a vicenda, e monitorare la loro salute diventa incredibilmente complesso con decine o centinaia di microservizi "
Ambienti coerenti ★	"L'ambiente di sviluppo e test deve essere il più vicino possibile alla produzione , per evitare problemi del tipo "funziona sulla mia macchina" "

La soluzione

"Gestire manualmente queste sfide per una grande distribuzione di microservizi non è semplicemente fattibile."

"Come soluzione, impareremo:"

- **strategie di deployment con container e orchestratori**
 - **pattern Kubernetes**
 - **pipeline DevOps e CI/CD seguendo l'Infrastructure as Code**
-